

בלאק-ג'ק רב-משתתפים - תיק פרויקט

חלופת התמחות: הגנת סייבר ומערכות הפעלה

מסלול: הנדסת תוכנה 883589

1. מבוא

1.1 ייזום

1.1.1 תיאור ראשוני של המערכת

הפרויקט הוא משחק בלאק-ג'יק (21) רב-משתתפים על רשת בארכיטקטורת לקוח-שרת. מספר שחקנים מתחברים ממחשבים שונים לשרת מרכזי, ומשחקים יחד כנגד דילר ממוחשב.

המוצר המוגמר כולל:

- שרת משחק - מקבל חיבורים, מנהל את חוקי המשחק, שומר פרופילי שחקנים ושומר נתונים לדיסק.
- לקוח גרפי (Tkinter) - מסך התחברות/הרשמה, שולחן בלאק-ג'יק עם קלפים, כפתורי - שדה הימור, סרגל פרופיל וצ'אט בזמן אמת, Hit/Stand/Double.
- לוח בקרה לשרת (ServerApp) - חלון ניהול נפרד: רשימת שחקנים מחוברים, כפתור Kick, לוג - צבעוני, סטטוס משחק ומונה הודעות Gemini.
- סינון צ'אט בינה מלאכותית (ChatModerator) - שולח כל הודעת צ'אט ל-Gemini 2.5 Flash - לפני שידורה. אם המפתח לא מוגדר, המשחק ממשיך לעבוד רגיל.
- שכבת הצפנה - לחיצת יד RSA + Fernet: כל הודעה מוצפנת ומאומתת.
- שכבת הרשאות - סיסמאות מגובבות (PBKDF2), כל פעולה מחייבת הזדהות עם השרת.
- מדוע בחרתי בפרויקט זה: בלאק-ג'יק פשוט מספיק בחוקיו כדי שהמיקוד יישאר בנושאי החלופה - תקשורת, ריבוי לקוחות, שמירת מצב והגנת סייבר - אך מורכב מספיק כדי לדרוש מכונת מצבים, סנכרון בין שחקנים ולוגיקה של קלפים.

אתגרים שצפיתי:

- תכנון פרוטוקול תקשורת שמתמודד עם הזרם הבלתי מוגדר של TCP.
- ניהול ריבוי לקוחות במקביל בעזרת threads בלי race conditions.
- מימוש נכון של לחיצת יד הצפנה אסימטרית-סימטרית וגיבוב סיסמאות.
- בניית מכונת מצבים שלא מאפשרת פעולה מחוץ לתור.

1.1.2 הגדרת הלקוח

המערכת מיועדת לשני קהלים:

- משתמשי קצה - נערים ומבוגרים שרוצים משחק קלפים חברתי ברשת מקומית, בלי תשלום אמיתי.
- מורים ותלמידים בחלופת סייבר - להדגמת עקרונות תקשורת מאובטחת, ריבוי לקוחות ואחסון מאובטח.

1.1.3 יעדים ומטרות

- לאפשר למספר שחקנים לשחק בלאק-ג'יק בו-זמנית סביב שולחן אחד.
- לאפשר הרשמה, התחברות ושמירת פרופיל (קרדיטים, ניצחונות, הפסדים, יחס W/L).
- להבטיח שכל התקשורת בין הלקוח לשרת מוצפנת ומאומתת.
- למנוע לקוח זדוני מלהשפיע על שחקנים אחרים או לזייף יתרה.

- ממשק משתמש ברור ונוח.
- שמירת נתוני משתמשים בין הפעלות שרת.

1.1.4 בעיות, תועלות וחסכוניות

- הבעיה: משחקי קלפים מקוונים קיימים הם בדרך כלל "קופסה שחורה" סגורה או דורשים תשתית ענן יקרה.
- תועלות:
 - חוויית משחק חברתית מקומית, בלי הרשמה לשירות חיצוני.
 - סביבת לימוד להבנת הצפנה, threads ו-sockets.
 - פרופילי משתמש שנשמרים בין הפעלות.
 - שירותים: הרשמה/התחברות, הימור, חלוקת קלפים, פעולות שחקן (Hit/Stand/Double), משחק - דילר, חישוב תוצאות ותשלום, צ'אט, רענון מצב שולחן בזמן אמת.

1.1.5 סקירת פתרונות קיימים

חסרונות מבחינתנו	יתרונות	מערכת קיימת
סגור, דורש תשלום, אין גישה לקוד	עיצוב מקצועי	Online Casino (888, PokerStars)
חד-שחקן, ללא רשת, ללא אבטחה	קוד פתוח, פשוט	Blackjack ב-Python (CLI)
מסתיר את שכבות ה-TCP, לא מלמד sockets	רב-משתתפים, נגיש	WebSocket Multiplayer בדפדפן

הפתרון שלי שונה בכך שהוא רב-משתתפים אמיתי על TCP גולמי, עם לחיצת יד מוצפנת שכתובה ידנית ושמירת פרופילים ב-SQLite מקומי - כך שכל שכבה ניתנת לקריאה ולהבנה.

1.1.6 סקירת טכנולוגיית הפרויקט

הפרויקט כולו ב-Python 3.11. רוב הספריות סטנדרטיות (socket, threading, sqlite3, struct, cryptography (RSA + Fernet), google-genai (Gemini AI צ'אט), secrets, hmac, hashlib, tkinter).

מגבלות:

- מיועד לרשת מקומית אמינה; לא לאינטרנט פתוח (אין תעודת CA, אין הגנת DDoS מתוככמת).
- דורש מערכת הפעלה עם תמיכה גרפית Tkinter מבוסס GUI

1.1.7 תיחום הפרויקט

מה הפרויקט כולל:

- תקשורת TCP sockets ובניית פרוטוקול מותאם אישית.
- ריבוי לקוחות בעזרת threads.
- הצפנה היברידית (RSA-2048 + Fernet/AES-128) ולחיצת יד.
- גיבוב סיסמאות (PBKDF2-HMAC-SHA256 עם salt).
- שמירת נתונים במסד נתונים SQLite.
- ממשק משתמש גרפי ב-Tkinter.
- מכונת מצבים למשחק רב-שלבי.

מה הפרויקט לא כולל:

- תשלומים אמיתיים או כסף.
- אחסון בענן / שרתים מבוזרים.
- תאימות לדפדפן / WebSockets / מובייל.
- גרפיקה תלת-ממדית.

1.2 פירוט תיאור המערכת (אפיון)

1.2.1 תיאור מפורט של המערכת

המערכת בנויה משני יישומים:

- שרת בלאק-ג'יק - בריצה ראשונה יוצר זוג מפתחות RSA, מאזין על פורט 5050, ומנהל חיבורי - שחקנים בתהליכים נפרדים.
- לקוח בלאק-ג'יק - חלון Tkinter שמתחבר לשרת, מבצע לחיצת יד מוצפנת, ומציג מסך התחברות - ומסך שולחן.

1.2.2 יכולות לפי סוג משתמש

יש סוג משתמש אחד - שחקן. יכולותיו:

- הרשמה (יצירת פרופיל עם 10,000 קרדיטים).
- התחברות.
- הצטרפות לשולחן.
- הימור בכל סבב.
- Hit, Stand, Double.
- צפייה בקלפי הדילר ושחקנים אחרים.
- צפייה בפרופיל ובסטטיסטיקה.
- צ'אט עם שחקנים אחרים.
- התנתקות.

1.2.3 בדיקות מתוכננות (קופסה שחורה)

אופן הביצוע	מטרת הבדיקה	#
לעצור ולפעיל שוב - קרדיטים וניצחונות נשמרו	שחזור פרופיל אחרי restart	10
לסגור לקוח בזמן תורו - התור עובר הלאה אוטומטית	מחזור באמצע סבב	11
לא קריאות handshake על הפורט - חבילות לא	handshake	8
ללחוץ Hit כשזה לא התור שלך - נדחה בשרת	פעולה מחוץ לתור	7
לבקש הימור גדול מהקרדיטים - שגיאה, היתרה לא	הימור מעל יתרה	6
הימור, Hit/Stand, דילר, תשלום נכון	משחק מלא לשני שחקנים	5
להתחבר עם "alice" פעמיים - החיבור השני נדחה	התחברות כפולה	4
להזין סיסמה לא נכונה - לקבל הודעה גברית	סיסמה שגויה	3
לרשום "alice" פעמיים - לקבל שגיאה בפעם השנייה	רשום כפול	2
לעצור ולפעיל שוב - קרדיטים וניצחונות נשמרו	שחזור פרופיל אחרי restart	10
לסגור לקוח בזמן תורו - התור עובר הלאה אוטומטית	מחזור באמצע סבב	11
לעצור ולפעיל שוב - קרדיטים וניצחונות נשמרו	שחזור פרופיל אחרי restart	10
לסגור לקוח בזמן תורו - התור עובר הלאה אוטומטית	מחזור באמצע סבב	11

לנסות שם עם תווים מיוחדים - נדחה	ולידציית שם משתמש	12
----------------------------------	-------------------	----

1.2.4 לוח זמנים

משך בפועל	משך מתוכנן	אבן דרך	#
שבוע 1-2	שבוע 1-2	אפיון, לימוד sockets ו-threads	1
שבוע 3-5 (חריגה בגלל debug של encoding)		פרוטוקול ולחיצת יד	2
שבוע 6-7	שבוע 5-6	מנוע משחק (Card/Deck/Hand/Table)	3
שבוע 7-8	שבוע 7	ניהול פרופילים והצפנת סיסמאות	4
שבוע 8-9	שבוע 8	שילוב שרת ולקוח, threads, broadcasts	5
שבוע 9-11	שבוע 9-10	GUI ב-Tkinter	6
שבוע 11-12	שבוע 11	בדיקות אינטגרציה ותיקון באגים	7
שבוע 12	שבוע 12	כתיבת תיקי הפרויקט	8

1.2.5 ניהול סיכונים

מה בוצע	תכנון	סיכון
with כל פעולה תחת Table ו-ProfileManager; רעל RLock משחק	RLock	Race conditions
החיבור נסגר בנימוס; הלקוח מציג "failed to connect" חיבור + try/except		כשל בלחיצת היד
ממומש ב-ProfileManager עם WAL_mode=PRAGMA journal_mode=WAL	PRAGMA journal_mode=WAL	שחיתות נתונים בכתיבה
ממומש ב-protocol.py	מגבלת 64 KB לפני קריאה	לקוח שולח הודעה ענקית
ממומש ב-AppController.enqueue_message	queue + root.after(0,...)	Tkinter לא thread-safe

1.3 תחום הידע - פירוט יכולות

1.3.1 יכולות בצד השרת

יכולת: קבלת חיבור חדש וניהולו בתהליכון נפרד

- מהות: השרת מאזין על פורט קבוע. לכל לקוח נפתח ClientHandler נפרד. -
- פעולות: socket.accept, threading.Thread, socket, ניהול רשימת מטפלים, סגירה מסודרת. -
- אובייקטים: BlackjackServer, ClientHandler. -

יכולת: לחיצת יד הצפנה (RSA → Fernet)

- מהות: השרת שולח מפתח ציבורי, הלקוח מגריל מפתח Fernet, מצפין אותו ב-RSA ושולח. מכאן - כל הודעה מוצפנת סימטרית.
- פעולות: טעינת/יצירת זוג RSA, פענוח OAEP, יצירת SecureChannel. -
- אובייקטים: RSAKeyPair, SecureChannel. -

יכולת: רישום משתמש חדש

- מהות: קליטת שם וסיסמה, ולידציה, גיבוב ושמירה. -

- פעולות: ולידציה, INSERT, hash_password (PBKDF2) לטבלת users ב-SQLite, commit.
- אובייקטים: ProfileManager, UserProfile.

יכולת: התחברות משתמש קיים

- מהות: השוואת סיסמה כנגד הגיבוב השמור, עמידות ב-timing attacks, מניעת התחברות כפולה.
- פעולות: verify_password עם hmac.compare_digest; השהיה דמה אם המשתמש לא קיים.
- אובייקטים: ProfileManager, ClientHandler.

יכולת: ניהול שולחן הבלאק-ג'ק

- מהות: מכונת מצבים WAITING → BETTING → PLAYING → DEALER → RESULTS.
- פעולות: קבלת שחקן, קבלת הימור, חלוקת קלפים, מעבר תור, משחק דילר, חישוב תוצאות, איפוס.
- אובייקטים: Table, Player, Dealer, Deck, Hand, Card, Phase.

יכולת: שידור מצב השולחן (broadcast)

- מהות: לאחר כל אירוע, השרת בונה תמונת מצב לכל שחקן (מסתיר קלף נעלם של דילר) ושולח לו.
- פעולות: סריאליזציה של Table, החלפת קלף נעלם, איטרציה על ClientHandler-ים.
- אובייקטים: BlackjackServer, Table.snapshot_for, ClientHandler.send.

יכולת: תשלום וזיכוי לפי תוצאת הסבב

- מהות: השוואת ידיים, תשלום (1:1, 3:2 לבלאק-ג'ק, החזר בפוש), עדכון פרופיל.
- פעולות: Table._resolve_locked, ProfileManager.adjust_credits, record_result.
- אובייקטים: Table, ProfileManager.

יכולת: סינון הודעות צ'אט (Gemini AI)

- מהות: לפני שידור הודעת צ'אט, נשלחת ל-Gemini 2.5 Flash. אם נחסמה – רק השולח מקבל שגיאה. אם ה-API לא זמין – ההודעה עוברת.
- פעולות: בדיקת GEMINI_API_KEY, קריאת google.genai, פרשנות ALLOW/BLOCK.
- אובייקטים: ChatModerator, ClientHandler.

יכולת: לוח בקרה גרפי לשרת

- מהות: חלון Tkinter בזמן אמת: IP/פורט, רשימת שחקנים, Kick, לוג, שלב משחק, מונה.
- עיצירת שרת, Gemini.
- פעולות: Tk window, root.after () כל 400ms, גישה ל-online_usernames תחת lock.
- אובייקטים: ServerApp, _PlayerRow, BlackjackServer.

1.3.2 יכולות בצד הלקוח

יכולת: התחברות לשרת ולחיצת יד

- מהות: פתיחת socket, פענוח server_hello, יצירת מפתח Fernet, הצפנה ב-RSA ושליחה.
- פעולות: socket.connect, serialization, SecureChannel.generate, encrypt_session_key.

- אובייקטים: BlackjackClient, SecureChannel.

יכולת: מסך התחברות/הרשמה

- מהות: שדות שם וסיסמה, שני כפתורים, הצגת תשובת שרת.
- פעולות: ttk.Frame, submit, טיפול בשגיאה.
- אובייקטים: LoginFrame, AppController.

יכולת: מסך השולחן

- מהות: אזור דילר, אזור שחקנים, פרופיל אישי, שדה הימור, כפתורי פעולה.
- פעולות: ציור קלף, סימון "(you)", איפסור/נטרול כפתורים לפי שלב.
- אובייקטים: TableFrame.

יכולת: רענון אסינכרוני של מצב השולחן

- מהות: הודעות מהרשת מגיעות ב-thread רקע. נדחפות לתור, ה-GUI מנקה ב-(...,root.after(0)).
- פעולות: queue.Queue, callback, _drain_messages.
- אובייקטים: AppController.

יכולת: שליחת פעולה (Hit/Stand/Double/Bet/Chat)

- מהות: לחיצה מתורגמת להודעה מוצפנת לשרת.
 - פעולות: ולידציה לפני שליחה, BlackjackClient.send, טיפול בכשל רשת.
 - אובייקטים: TableFrame, AppController, BlackjackClient.
-

2. מבנה / ארכיטקטורה של הפרויקט

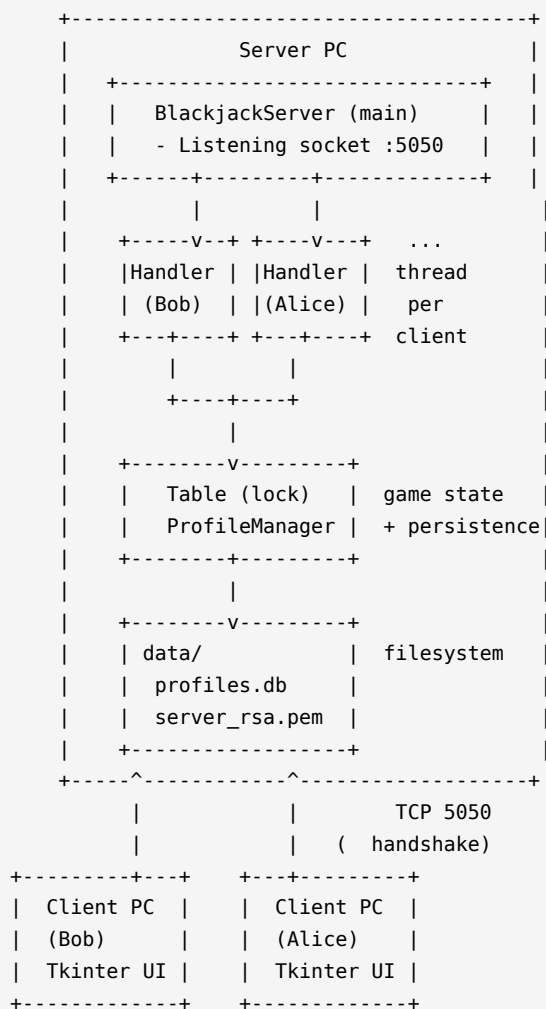
2.1 ארכיטקטורה כללית

המערכת בארכיטקטורת לקוח-שרת עם שרת מרכזי אחד. כל הקלפים, הקופה והכללים מנוהלים על ידי השרת. הלקוחות מציגים מצב ושולחים בקשות פעולה - הם לא מקור האמת.

2.1.1 רכיבי חומרה

- מחשב שרת: מריץ `python -m blackjack.run_server`. צריך Python 3.11, חיבור רשת, מקום - לשני קבצים (`profiles.db`, `server_rsa.pem`).
- מחשבי לקוח: כל שחקן מריץ `python -m blackjack.run_client` באותה רשת.
- רשת: TCP/IPv4 על פורט 5050.

2.1.2 שרטוט ארכיטקטורה



2.2 טכנולוגיות בשימוש

סיבה	טכנולוגיה	תחום
קריאה, ספריית סטנדרט עשירה	Python 3.11	שפת תכנות
אמין, מאפשר פרוטוקול מותאם	גולמי, פורט 5050 TCP	תקשורת
תקן מקובל להעברת מפתח	RSA-2048 + OAEP	הצפנה אסימטרית
סודיות + אימות שלמות	Fernet (AES-128-CBC + HMAC-SHA256)	הצפנה סימטרית
קשה ל-brute-force	איטרציות 200,000 PBKDF2-HMAC-SHA256	גיבוב סיסמאות
בנוי ב-Python, ללא תלות	Tkinter	ממשק משתמש
עמיד בפני קריסה, תומך בשאילתות SQL, חלק מ-WAL mode	SQLite (sqlite3) - WAL mode	אחסון
פשוט ומספיק יעיל	threading.Thread + RLock	ריבוי לקוחות
מבין הקשר ושפות	Gemini 2.5 Flash (google-genai)	סינון צ'אט

2.3 זרימת מידע

2.3.1 חיבור שחקן

[Client]	[Server]
TCP connect :5050	
----->	
server_hello {public_key_pem}	
<-----	
key_exchange {encrypted_session_key}	
----->	
ready (encrypted)	
<-----	
login {username, password} (encrypted)	
----->	
	[authenticate]
auth_result {ok, profile} (encrypted)	
<-----	
game_state {snapshot} (encrypted)	
<-----	

2.3.2 זרימת סבב

BETTING:

```
client --place_bet 200--> server --> Table, ProfileManager
server --> broadcast game_state --> all clients
```

PLAYING:

```
Table.deal() -> 2 cards each, dealer hidden card
server -> snapshot for each client
Loop: client --action {hit|stand|double}--> server --> Table
server -> broadcast
```

DEALER:

```
server reveals dealer hole card
while dealer.should_hit(): dealer.add(deck.draw())
server -> broadcast
```

RESULTS:

```
compute outcome, payout, record W/L
server -> round_result + updated profile
Timer (6s) -> next round
```

2.4 אלגוריתמים מרכזיים

2.4.1 חישוב ערך יד עם אסים גמישים

בעיה: אם שווה 1 או 11 - יש לבחור את הצירוף המרבי שלא חוצה 21.

הערות	סיבוכיות	פתרון
יקר	$O(2^k)$	בדיקה רקורסיבית של כל הצירופים
פשוט, מקובל בקזינו	$O(n)$	ספירת אסים כ-1, "שדרוג" כל עוד לא חוצה 21

נבחר: הפתרון השני. ראה `Hand.value` → `common/hand.py`.

2.4.2 ערבוב מאובטח

בעיה: סדר קלפים שלא ניתן לחיזוי.

בעיה	פתרון
ניתן לחיזוי לאחר 624 ערכים - Mersenne Twister	<code>random.shuffle</code>
מקור אקראי קריפטוגרפי, לא ניתן לחיזוי	Fisher-Yates + <code>secrets.randbelow</code>

נבחר: Fisher-Yates עם `secrets.randbelow`. ראה `Deck.resuffle` → `common/deck.py`.

2.4.3 גיבוב סיסמאות PBKDF2

בעיה: אחסון בטוח שלא יאפשר שחזור אם הקובץ ייגנב.

בעיה	פתרון
------	-------

פשוט MD5/SHA-1	מהיר מדי, פגיע למילון ו-rainbow tables
PBKDF2 + salt	200,000 איטרציות, כל ניחוש דורש עבודה יקרה

נבחר: 200,000 PBKDF2-HMAC-SHA256, salt איטרציות, ייחודי לכל משתמש. חלק מ-hashlib הסטנדרטית. ראה `common/crypto.py`.

2.4.4 מסגור הודעות מעל TCP

בעיה: TCP מספק זרם בתים, לא הודעות.

יתרונות/חסרונות	פתרון
עלול להופיע בתוכן	delimiter (n\)
בינארי בטוח, פשוט, ניתן להגביל גודל	בתים 4 length-prefix

נבחר: 4 length-prefix big-endian בתים + מגבלת 64 KB. ראה `common/protocol.py`.

2.4.5 סינון צ'אט בזמן אמת עם Gemini AI

בעיה: שחקנים עשויים לשלוח הודעות פוגעניות. רשימת מילים שחורות לא מספיקה - מחמיצה ניסוחים יצירתיים.

חסרון	פתרון
קל לעקוף, לא מבין הקשר	מילים Blocklist
תלות נוספת	Perspective API
מבין הקשר, עברית ואנגלית, מהיר	Gemini 2.5 Flash

נבחר: Gemini 2.5 Flash עם ה-system prompt:

"Useful, sexually explicit, racist, threatening, or otherwise offensive content. Reply with exactly one word: ALLOW or BLOCK."

אם Gemini מחזיר BLOCK - השרת שולח שגיאה רק לשולח. אם ה-API לא זמין - ההודעה עוברת (pass-through mode).

קובץ: `(server/chat_moderator.py → ChatModerator → check(message))`.

```
allowed, reason = _moderator.check(text)
if not allowed:
    self.send("error", {"message": reason})
else:
    self._server.handle_chat(self, text)
```

2.5 סביבת פיתוח

- Python עם תוסף VS Code, Python 3.11
- ניהול חבילות: pip.
- חבילות: cryptography, google-genai.

בדיקות: הרצה ידנית של שרת ושני לקוחות; tcpdump לידוא הצפנה; ls -l לבדיקת הרשאות. -

2.6 פרוטוקול תקשורת

2.6.1 שכבות הפרוטוקול

- שכבת מסגור: 4 בתים big-endian (אורך) + n בתים גוף. מגבלה: $0 < 65,536 \leq n$.
- שכבת הצפנה: עד סיום JSON - handshake גלוי. לאחר מכן - פלט Fernet (כולל HMAC).
- שכבת הודעה: JSON עם המבנה {"type": "...", "data": {...}}. סוג ההודעה חייב להיות - מתוך רשימה סגורה.

2.6.2 פירוט ההודעות

שדות	מ→אל	שם הודעה
public_key_pem: str	שרת → לקוח	server_hello
encrypted_session_key: str (base64)	לקוח → שרת	key_exchange
{}	שרת → לקוח	ready
username: str, password: str	לקוח → שרת	register
username: str, password: str	לקוח → שרת	login
ok: bool, message: str, profile?: dict	שרת → לקוח	auth_result
amount: int	לקוח → שרת	place_bet
action: "hit" / "stand" / "double"	לקוח → שרת	action
base, dealer, players, current_username, deck_remove	שרת → לקוח	game_state
outcome: win/loss/push, payout: int, profile: dict	לקוח → שרת	round_result
Gemini עד 200 תווים), עובר סינון) message: str	לקוח → שרת → שרת → כולם	chat
message: str	שרת → לקוח	error
{}	לקוח → שרת	logout

2.7 מסכי המערכת

2.7.1 מסך התחברות / הרשמה

- שדה Username, שדה Password (תווים מוסתרים), כפתורי Login ו-Register, תווית מצב. -

2.7.2 מסך השולחן

- סרגל פרופיל עליון (קרדיטים, ניצחונות, הפסדים, W/L), אזור דילר, אזור שחקנים (הדגשה - לשחקן עצמו), שדה הימור, כפתורי Hit/Stand/Double, צ'אט בצד.

2.7.3 תוצאת סבב

באנר (WIN = זיה, LOSS = אדום, PUSH = אפור) למשך 6 שניות ואז סבב חדש. -

2.7.4 לוח בקרה לשרת

- כותרת: שם האפליקציה + Host/IP/Port.
- שמאל: רשימת שחקנים מחוברים + כפתור Kick לכל אחד.
- ימין: לוג עם צבע לפי חומרה.
- שורה תחתונה: שלב משחק, קלפים שנותרו, מונה Gemini, כפתור Stop.

2.7.5 תרשים מסכים

```

+-----+ login OK +-----+
| LoginFrame |----->| TableFrame |
+-----+ +-----+
^
| disconnect / logout |
+-----+

+-----+ ( )
| ServerApp |
+-----+

```

2.8 מבני נתונים

2.8.1 טבלת users - מסד הנתונים profiles.db

מסד הנתונים הוא קובץ SQLite יחיד. הטבלה נוצרת אוטומטית בריצה ראשונה.

```

CREATE TABLE IF NOT EXISTS users (
  username      TEXT      PRIMARY KEY,
  password_hash TEXT      NOT NULL,
  credits       INTEGER NOT NULL DEFAULT 10000,
  wins          INTEGER NOT NULL DEFAULT 0,
  losses        INTEGER NOT NULL DEFAULT 0,
  pushes        INTEGER NOT NULL DEFAULT 0
);

```

דוגמה	אורך / טווח	טיפוס	שם עמודה
"alice"	3-16 תווים, _A-Za-z0-9	TEXT (PRIMARY KEY)	username
"k4Ja...\$g2Cv..."	פורמט salt_b64\$hash_b64	TEXT	password_hash
9750	>= 0	INTEGER	credits
4	>= 0	INTEGER	wins
2	>= 0	INTEGER	losses
1	>= 0	INTEGER	pushes

כתיבות אוטומיות מובנות, עמיד בפני קריסה - WAL (Write-Ahead Log) פועל במצב SQLite

2.8.2 קובץ server_rsa.pem

מפתח RSA פרטי בפורמט PEM. הרשאות 0600 (קריאה/כתיבה לבעל בלבד). נוצר אוטומטית בריצה ראשונה.

2.8.3 מבני נתונים בזיכרון

- BlackjackServer._clients : List[ClientHandler]
- BlackjackServer._online : Dict[str, ClientHandler]
 - Table._players : List[Player]
 - Deck._cards : List[Card]
- ApplicationController._queue : queue.Queue

2.9 סקירת חולשות ואיומים**2.9.1 שכבת האפליקציה**

פתרון	רלוונטי?	איום
כל שאילתה משתמשת בפרמטרים (?) ולא בשרשור SQL	כן	SQL Injection
הודעה גנרית; ה; salt; hmac.compare_digest	כן	אימות (login)
שינוי בית אחד זורק חריגה; RSA-2048 + OAEP + Fernet	כן	MITM
מגבלת 64 thread; KB לכל לקוח; socket timeout	כן	DoS
סוגר חיבו VALID_TYPES; ProtocolError; קפדן parseq	כן	Fuzzing
מסתיר קלף נעלם; snapshot_for; require_auth	כן	גישה לנתוני משתמש אחר
random לא, seqs.randbelow (/dev/urandom)	כן	חלש Randomness
רק כמשתנה סביבה; לא בקוד ולא בלוג	כן	חשיפת מפתח Gemini

2.9.2 שכבת התעבורה

- לחיצת יד משולשת מנוהלת על ידי מערכת ההפעלה - TCP
- הצפנה: (hybrid handshake (RSA + Fernet) - סודיות + שלמות ללא TLS.
- מבטיח שלא נקרא חצי הודעה recv_exact

3. מימוש הפרויקט

3.1 חלק א' - תיאור מחלקות וספריות

3.1.1 ספריות חיצוניות

שימוש	ספרייה
RSA-2048 + OAEP	<code>cryptography.hazmat.primitives.asymmetric.rsa</code>
פורמט PEM	<code>cryptography.hazmat.primitives.serialization</code>
הצפנה סימטרית מאומתת	<code>cryptography.fernet.Fernet</code>
קריאות ל-Gemini 2.5 Flash לסינון צ'אט	<code>google.genai</code>

3.1.2 מחלקות שפותחו בפרויקט

Card (common/card.py)

- תפקיד: קלף בודד, `immutable`.
- תכונות: `rank: str ('A','2'..'K')`, `suit: str ('S','H','D','C')`.
- פעולות: ולידציה; `to_dict/from_dict`; `base_value`; `is_ace`.

Deck (common/deck.py)

- תפקיד: "shoe" של N חבילות מעורבבות.
- תכונות: `cards: list[Card]`, `num_decks: int`.
- פעולות: `__len()`; `draw() → Card`; `secrets.randbelow`; `Fisher-Yates`; `__reshuffle()`.

Hand (common/hand.py)

- תפקיד: יד של שחקן או דילר עם חישוב ערך חכם.
- פעולות: `add(card)`; `clear()`; `value (property)`; `is_soft`; `is_blackjack`; `is_bust`; `to_list()`.

SecureChannel (common/crypto.py)

- תפקיד: עטיפה ל-Fernet לערוץ סימטרי.
- פעולות: `generate()` (classmethod); `encrypt(plaintext)`; `decrypt(token)`.

RSAPublicKey (common/crypto.py)

- תפקיד: מפתחות RSA-2048 של השרת.
- פעולות: `generate()`; `load_or_create(path)`; `public_pem()`; `decrypt_session_key(b64)`.

UserProfile (server/profile_manager.py)

- תפקיד: רשומת משתמש.
- תכונות: `username`, `password_hash`, `credits`, `wins`, `losses`, `pushes`.
- פעולות: `games_played`; `wl_ratio`; `to_public_dict`.

ProfileManager (server/profile_manager.py)

תפקיד: ממשק לבסיס הנתונים (SQLite (profiles.db; רישום, אימות, שינוי קרדיטים, עדכון - תוצאות.

- `.conn: sqlite3.Connection, _lock: RLock_`: תכונות:

- `.register(u,p); authenticate(u,p); adjust_credits; record_result; get`: פעולות:

Phase (server/game.py)

תפקיד: enum של חמשת השלבים - .WAITING, BETTING, PLAYING, DEALER, RESULTS

Table (server/game.py)

תפקיד: מכונת מצבים של שולחן בלאק-ג'ק. -

- `.players, _dealer, _deck, _phase, _lock`: RLock, `callbacks_`: תכונות:

- ;add_player; place_bet; player_action; _start_round_locked: פעולות:
_run_dealer_locked; _resolve_locked; snapshot_for(username).

ClientHandler (server/client_handler.py) - thread

תפקיד: thread אחד לכל לקוח.

- ;()run(); _handshake(); _main_loop(); _dispatch(); _handle_login/register פעולות: send(); shutdown().

BlackjackServer (server/server.py)

תפקיד: listening socket, accept loop, broadcasts, תזמון סבב.

- ;serve_forever(); stop(); on_client_authenticated; on_client_disconnected: פעולות: handle place bet; handle action; handle chat; broadcast state.

BlackjackClient (client/network.py)

תפקיד: שכבת רשת בלקוח כולל thread קבלה.

- `.()connect(); handshake(); send(); recv loop(); close` פעולות:

AppController (client/gui.py)

תפקיד: בקר ראשי - מחבר בין רשת ל-GUI. -

- ;run(); send(); do_login; do_register; do_logout; _enqueue_message : פעולות
drain messages; handle message; show login/ show table.

ChatModerator (server/chat_moderator.py)

- תפקיד: סורק הודעות צ'אט עם Gemini.

- `check(message) → (allowed, reason)` פעולות:

ServerApp (server/gui.py)

תפקיד: לוח בקרה גרפי לשרת. -

- `poll: פעולות: run() כל refresh players on stop(); kick player(); 400ms; .()`

3.2 קטעי קוד מרכזיים

3.2.1 חישוב ערך יד עם אסים


```
# blackjack/common/hand.py
@property
def value(self) -> int:
    total = sum(c.base_value for c in self._cards) # all aces = 1
    aces = sum(1 for c in self._cards if c.is_ace)
    while aces > 0 and total + 10 <= 21:
        total += 10 # promote one ace from 1 to 11
        aces -= 1
    return total
```

3.2.2 ערבוב מאובטח של החבילה

```
# blackjack/common/deck.py
def reshuffle(self) -> None:
    self._cards = [
        Card(rank, suit)
        for _ in range(self.num_decks)
        for suit in Card.SUITS
        for rank in Card.RANKS
    ]
    # Fisher-Yates with secrets.randbelow
    for i in range(len(self._cards) - 1, 0, -1):
        j = secrets.randbelow(i + 1)
        self._cards[i], self._cards[j] = self._cards[j], self._cards[i]
```

3.2.3 לחיצת יד הצפנה (צד שרת)

```
# blackjack/server/client_handler.py
def _handshake(self) -> None:
    send_plain(self._sock, "server_hello",
               {"public_key_pem": self._rsa.public_pem()})
    msg = recv_plain(self._sock)
    if msg["type"] != "key_exchange":
        raise ProtocolError("expected key_exchange")
    session_key = self._rsa.decrypt_session_key(
        msg["data"]["encrypted_session_key"])
    self._channel = SecureChannel(session_key)
    send_encrypted(self._sock, self._channel, "ready", {})
```

3.2.4 גיבוב והשוואת סיסמאות

```
# blackjack/common/crypto.py
ITERATIONS = 200_000

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    return f"{base64.b64encode(salt).decode()}$" \
        f"{base64.b64encode(digest).decode()}"

def verify_password(password: str, stored: str) -> bool:
    try:
        salt_b64, hash_b64 = stored.split("$", 1)
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(hash_b64)
    except (ValueError, binascii.Error):
        return False
    actual = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    return hmac.compare_digest(actual, expected) # constant-time
```

3.2.5 מסגור הודעה עם הגנה על גודל

```
# blackjack/common/protocol.py
MAX_MESSAGE_BYTES = 64 * 1024
_LENGTH_PREFIX = struct.Struct(">I")

def send_frame(sock, payload):
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ProtocolError("payload too large")
    sock.sendall(_LENGTH_PREFIX.pack(len(payload)) + payload)

def recv_frame(sock):
    header = _recv_exact(sock, _LENGTH_PREFIX.size)
    (length,) = _LENGTH_PREFIX.unpack(header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ProtocolError(f"invalid frame length: {length}")
    return _recv_exact(sock, length)
```

3.2.6 צילום מצב בטוח לכל שחקן

```
# blackjack/server/game.py
def snapshot_for(self, username: str) -> dict:
    with self._lock:
        dealer_cards = self._dealer.hand.to_list()
        if self._phase is Phase.PLAYING and len(dealer_cards) >= 2:
            dealer_cards = [dealer_cards[0], {"rank": "?", "suit": "?"}]
            value_hidden = True
        else:
            value_hidden = False
    return {
        "phase": self._phase.value,
        "dealer": {"cards": dealer_cards,
                   "value_hidden": value_hidden,
                   "value": self._dealer.hand.value
                   if not value_hidden else None},
        "players": [self._player_dict(p, p.username == username)
                    for p in self._players],
        "current_username": (self.current_player().username
                             if self.current_player() else None),
        "deck_remaining": len(self._deck),
    }
```

3.3 בדיקות

3.3.1 בדיקות שתוכננו

בעיות ופתרון	תוצאה	מטרה	#
-	רשומה נוצרה ב-GUI; profiles.db	עבר שמור חקיקה	1
-	"username already taken"	מניעת רישום כפול	2
בגרסה ראשונה חשף "unknown user"	נכונה	valid username or password שגויה	3
-	חיבור א' לא הופרע; "login already logged in"	login already logged in	4
תשלום 3:2 לבלאק-ג'ק - תוקן מ-1:	1:1	משחק מלא לשניים	5
-	יתרה לא השתנתה; "insufficient credits"	insufficient credits	6
-	"Not your turn"	פעולה מחוץ לתור	7
-	handshake - רעש בינארי בלבד; האזנה לרשת	האזנה לרשת	8
מוסיף בדיקה לפני הקצאה recv_frame	נכונה	הודעה ענקית (KB 200)	9
-	קרדיטים ונתונים נשמרו	restart אחרי persistence	10
נוסף טיפול ב-client_disconnected	נכונה	ניתוק באמצע סבב	11
-	נדחה: "only letters, digits and underscore"	only letters, digits and underscore משתמש	12

3.3.2 בדיקות נוספות

בעיות ופתרון	תוצאה	מטרה	#
-	עבר לחלוטין	אוטומטי end-to-end	13
בתחילה 644; נוסף os.chmod לאחר (0o600) -rw-----	נכונה	הרשאות PEM	14

15	handshake שגוי אחרי JSON	ProtocolError; החיבור נסגר	-
16	10 לקוחות בו זמנית	handshake; CPU <5% עברו	-
17	secrets vs random	אין דפוס ניתן לחיזוי	-
18	Connection lost" רשת	חזרה למסך התחברות;	-

4. מדריך למשתמש

4.1 עץ קבצים

```

blackjack/
├── README.md
├── .md
├── run_server.py          <-  ()
├── run_server_gui.py     <-
├── run_client.py
├── common/
│   ├── card.py
│   ├── deck.py
│   ├── hand.py
│   ├── protocol.py
│   └── crypto.py
├── server/
│   ├── profile_manager.py
│   ├── game.py
│   ├── client_handler.py
│   ├── chat_moderator.py
│   ├── gui.py
│   └── server.py
├── client/
│   ├── network.py
│   └── gui.py
└── data/                  <-  (gitignored)
    ├── profiles.db
    └── server_rsa.pem
  
```

4.2 התקנה

4.2.1 דרישות

- (ומעלה) מומלץ Python 3.10 3.11
- Linux / macOS / Windows.
- פעיל pip
- רשת מקומית בה השרת והלקוחות יכולים לראות זה את זה.

4.2.2 התקנת חבילות

```
pip install cryptography google-genai
```

בלינוקס אפשר שצריך Python-כלול ב Tkinter

```
sudo apt install python3-tk
```

סינון צ'אט Gemini הוא אופציונלי. ללא GEMINI_API_KEY כל הודעה עוברת. להפעלה:

```
export GEMINI_API_KEY="< >" # Linux / macOS
set GEMINI_API_KEY=< > # Windows
```

4.2.3 נתונים התחלתיים

- אין משתמש ברירת מחדל - כל שחקן נרשם בעצמו.
- קרדיטים ראשוניים: 10,000.
- גבולות הימור: 50 עד 5,000.

4.2.4 רשת

- ברירת מחדל: שרת מאזין על 0.0.0.0:5050; לקוח מתחבר ל-127.0.0.1:5050.
- לשינוי:

```
BJ_HOST=192.168.1.10 BJ_PORT=5050 python -m blackjack.run_client
```

4.2.5 דרישות חומרה מינימליות

מחשב 1 GB RAM, 512 MHz, פחות מ-5 MB מקום פנוי.

4.3 הפעלה

4.3.1 הפעלת השרת

א. מצב טרמינל:

```
python -m blackjack.run_server
```

ב. מצב לוח בקרה גרפי:

```
python -m blackjack.run_server_gui
```

לעצירה: Ctrl+C או כפתור Stop Server.

4.3.2 הפעלת הלקוח

```
python -m blackjack.run_client
```

נפתח חלון התחברות. יש להזין שם וסיסמה וללחוץ Register (פעם ראשונה) או Login.

4.3.3 תיאור מסכים

- מסך התחברות: "הקלד שם משתמש וסיסמה. אם עדיין לא נרשמת, לחץ Register".
- מסך שולחן - תחילת סבב: "הקלד הימור ולחץ Place Bet".
- מסך שולחן - תורך: "לחץ Hit לקלף נוסף, Stand לעצירה, Double להכפלת הימור".
- שלב דילר: "הקלף הנעלם נחשף; הדילר לוקח קלפים אוטומטית".
- תוצאה: "באנר זהב = ניצחת; אדום = הפסדת; אפור = פוש".
- לוח בקרה שרת: "רשימת שחקנים עם Kick, לוג, שלב משחק ומונה Gemini".

5. סיכום אישי / רפלקציה

עבודה על הפרויקט הייתה מסע ארוך. בתחילה היה לי רעיון מעורפל - "אעשה משחק קלפים עם רשת" - אבל מהר מאוד גיליתי שכל מילה מסתירה עולם של החלטות.

הצלחות:

הרגע שהפעלתי שני לקוחות בו-זמנית והם ראו אותו שולחן - threads הפכו מתיאוריה לכלי - עבודה.

כש-tcpdump הראה שהתעבורה לא קריאה אחרי handshake - ההצפנה עבדה. -
כשהבנתי לבד למה random לא מתאים לקזינו ועברתי ל-secrets. -

אתגרים:

הבדל בין bytes למחרוזות ב-Python 3 - שעות על TypeError. -
באותו רגע Hit בגרסה ראשונה לא תמיד ניתן היה להמר כשמישהו אחר שלח Race conditions -
הפתרון: threading.RLock על כל פעולה משנה-מצב.

למידה:

רוב הלמידה הייתה עצמאית - תיעוד Python, Real Python, Computerphile. -
בכל פעם שנתקעתי כתבתי לעצמי "מה בדיוק אני מנסה להעביר" - הטקסט הזה לבדו פתר חצי - מהבאגים.

הרגלים שאקח הלאה:

להגדיר פרוטוקול תקשורת בכתב לפני שכותבים שורת קוד. -
להשתמש ב-logging מסודר ולא ב-print. -
להפריד שכבות - common, server, client - כך שכל קובץ קריא לבד. -

במבט לאחור - מה הייתי משנה:

הייתי בוחר asyncio במקום threading לניהול ריבוי לקוחות. -
הייתי מוסיף pytest עם בדיקות יחידה מההתחלה. -

שאלות שנשארו פתוחות:

האם secrets.randbelow עמיד לחלוטין, או שיש מתקפות על urandom? -
איך עובד OAEP מתחת למכסה המנוע? -
איך קזינואים אמיתיים מוודאים שהדילר שלך קלף מהצמרת? -

6. ביבליוגרפיה

- Python Software Foundation. (2024). socket — Low-level networking interface.
<https://docs.python.org/3/library/socket.html>
 - Python Software Foundation. (2024). threading — Thread-based parallelism.
<https://docs.python.org/3/library/threading.html>
 - Python Software Foundation. (2024). secrets — Generate secure random numbers.
<https://docs.python.org/3/library/secrets.html>
 - Python Software Foundation. (2024). hashlib — Secure hashes and message digests.
<https://docs.python.org/3/library/hashlib.html>
 - Python Cryptographic Authority. (2024). Cryptography documentation.
<https://cryptography.io/en/latest/>
 - Bellare, M., & Rogaway, P. (1995). Optimal asymmetric encryption — How to encrypt with RSA. EUROCRYPT '94.
 - OWASP Foundation. (2023). Password Storage Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
 - Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson.
 - Computerphile. (2017). Public Key Cryptography [Video].
https://www.youtube.com/watch?v=GSIDS_lvRv4
 - Google. (2025). Google Generative AI Python SDK (google-genai).
<https://ai.google.dev/gemini-api/docs/sdks>
-

7. נספחים

נספח א' - הסבר טכנולוגיות

B.1 RSA-2048 + OAEP

הוא הצפנה אסימטרית - מפתח ציבורי ופרטי. כל מה שמוצפן עם הציבורי ניתן לפיענוח רק עם RSA הפרטי. משמש פעם אחת לכל חיבור להעברת מפתח OAEP. Fernet מוסיף אקראיות ומונע מתקפות על RSA.

B.2 Fernet

מפרט המבוסס על AES-128-CBC + HMAC-SHA256. כל שינוי בבית אחד מזוהה בפענוח. כולל timestamp שמגלה הודעות "ישנות".

B.3 PBKDF2

הפעלה חוזרת של HMAC על הסיסמה ו-200,000 salt. איטרציות הופכות כל ניחוש ליקר, מה שמקשה על brute-force.

B.4 TCP

מקנה זרם בתים אמין בין שני קצוות. אנחנו בונים מעליו פרוטוקול הודעות עם length-prefix.

B.5 Threads ו-Locks

אחד משנה נתון בזמן נתון thread מבטיח שרק RLock. הוא נתיב ביצוע נוסף Thread.

B.6 Gemini AI

מודל שפה גדול של Google. בפרויקט זה Gemini 2.5 Flash מסוג הודעות צ'אט כ-ALLOW או BLOCK. אם ה-API לא זמין, ChatModerator מאפשר מעבר כברירת מחדל.

נספח ג' - קוד מלא (לצירוף בגרסה מודפסת)

יש לצרף את הקבצים בסדר הבא:

- common/card.py
- common/deck.py
- common/hand.py
- common/protocol.py
- common/crypto.py
- server/profile_manager.py

- server/game.py
- server/client_handler.py
 - server/server.py
- client/network.py
 - client/gui.py
- server/chat_moderator.py
 - server/gui.py
- run_server.py
- run_server_gui.py
 - run_client.py